

Nama : Auliya Magfirani Arzal

Nim : D121251064

Modul 1 : Perkalian Matriks Standar ($O(n^3)$)

1. Apa peran variabel k pada loop terdalam? Data apa yang diakses dari matriks A dan matriks B pada setiap iterasi k?
k adalah penghubung antara baris dan kolom, yang dimana sebagai indeks untuk proses penjumlahan hasil kali. $C[i][j] += A[i][k] * B[k][j]$; Setiap nilai k diambil dari 1 elemen baris ke-i dari matriks A dan 1 elemen dari kolom ke-j dari B, kalikan lalu tambahkan ke hasilnya.
2. Kenapa kita harus membebaskan memori menggunakan free() setelah program selesai? Apa yang terjadi pada RAM komputer jika kita terus mengalokasikan memori tanpa pernah membebaskannya (Memory Leak)?
Karena saat kita menggunakan malloc(), program akan meminta ruang penyimpanan di RAM (heap). Berbeda dengan RAM (Stack) yang menyediakan memori secara otomatis dan akan terhapus ketika fungsi selesai, RAM (Heap) akan tetap menyimpan memori yang telah dialokasikan meskipun fungsi telah selesai dijalankan, sehingga memori tersebut harus dibebaskan secara manual menggunakan fungsi free(). Jika memori tidak dibebaskan, akan terjadi memory leak, yaitu kondisi di mana memori terus terpakai tanpa bisa digunakan kembali, sehingga dapat menyebabkan RAM penuh, program menjadi lambat, bahkan crash.
3. Bagaimana jumlah operasi perkalian berubah jika ukuran matriks n menjadi $2n$? Buktikan bahwa kompleksitasnya adalah $O(n^3)$.
Jika ukuran matriks berubah dari n menjadi $2n$, maka jumlah operasi perkalian menjadi $(2n)^3 = 8n^3$, yaitu meningkat 8 kali lipat. Namun dalam notasi Big-O, konstanta dapat diabaikan karena yang diperhatikan adalah laju pertumbuhan algoritma terhadap ukuran input. Konstanta tidak mempengaruhi bentuk pertumbuhan, sehingga nilai seperti $8n^3$ atau $100n^3$ tetap dianggap $O(n^3)$.

Modul 2 : Strassen Matrix Multiplication

1. Apa keuntungan menggunakan malloc dibandingkan array statis pada kasus ini?
Keuntungan menggunakan malloc dibandingkan array statis adalah ukuran memori dapat ditentukan secara dinamis saat runtime, lebih fleksibel dalam penggunaan, dapat menangani data berukuran besar karena menggunakan heap, serta memori dapat dibebaskan kembali menggunakan free(). Sedangkan array statis memiliki ukuran tetap dan terbatas pada stack.
2. Kenapa inisialisasi random penting untuk menguji validitas algoritma pada berbagai distribusi data?
Inisialisasi random penting karena dapat menguji algoritma pada berbagai distribusi data yang berbeda, sehingga hasil pengujian menjadi lebih objektif dan tidak bias. Selain itu, data acak dapat mensimulasikan kondisi nyata, membantu mendeteksi kesalahan tersembunyi, serta memastikan algoritma tetap bekerja dengan benar pada berbagai kemungkinan input.
3. Bagaimana pergerakan Stack Pointer saat fungsi strassen memanggil dirinya sendiri secara rekursif?
Saat fungsi strassen memanggil dirinya sendiri secara rekursif, Stack Pointer akan bergerak turun setiap kali terjadi pemanggilan fungsi karena stack frame baru ditambahkan. Ketika

fungsi selesai dan melakukan return, Stack Pointer akan bergerak naik karena stack frame dihapus. Semakin dalam rekursi, semakin banyak memori stack yang digunakan.

Modul 3 : Konvolusi 2D (Image Blurring)

1. Apa yang terjadi jika kita tidak melakukan pengecekan batas (boundary check) pada koordinat piksel?

Jika tidak dilakukan boundary check, program dapat mengakses memori di luar batas array gambar, yang dapat menyebabkan error seperti segmentation fault, hasil gambar menjadi tidak valid, atau muncul bug yang sulit dideteksi. Oleh karena itu, boundary check diperlukan untuk memastikan hanya piksel yang valid yang diproses.

2. Kenapa algoritma ini memiliki kompleksitas $O(N^2 \cdot K^2)$?

Algoritma ini memiliki kompleksitas $O(N^2 \cdot K^2)$ karena terdapat dua loop utama yang mengiterasi seluruh piksel gambar berukuran $N \times N$ (sehingga menghasilkan N^2), dan di dalamnya terdapat dua loop lagi yang mengiterasi kernel berukuran $K \times K$ (sehingga menghasilkan K^2). Oleh karena itu, total kompleksitasnya adalah $O(N^2 \cdot K^2)$.

3. Bagaimana cara memodifikasi algoritma ini agar lebih cepat menggunakan teknik Separable Convolution?

Algoritma dapat dipercepat menggunakan teknik Separable Convolution dengan memecah kernel 2D menjadi dua kernel 1D, yaitu horizontal dan vertikal. Proses dilakukan dalam dua tahap, pertama melakukan konvolusi horizontal, kemudian hasilnya diproses kembali dengan konvolusi vertikal. Dengan cara ini, kompleksitas berkurang dari $O(N^2 \cdot K^2)$ menjadi $O(N^2 \cdot K)$.